

BEP-007: Tuplas e Dicionários

Conheça outras estruturas de dados fundamentais em Python

Diferenças: Tuplas vs Listas

Comparação Detalhada

Entender as diferenças entre tuplas e listas é fundamental para escolher a estrutura de dados mais adequada para cada situação.

Análise Comparativa

Característica	Tupla	Lista	Impacto
Mutabilidade	 Imutável	 Mutável	Tuplas são mais seguras
Performance	 Mais rápida	 Mais lenta	Tuplas são mais eficientes
Memória	 Menos memória	 Mais memória	Tuplas economizam espaço
Métodos	count(), index()	append(), remove(), etc.	Listas têm mais funcionalidades
Hashable	 Sim	 Não	Tuplas podem ser chaves

Teste de Performance

```
import time
```



```
# Teste de criação
inicio = time.time()
tupla = tuple(range(1000000))
tempo_tupla = time.time() - inicio

inicio = time.time()
lista = list(range(1000000))
tempo_lista = time.time() - inicio

print(f"Tupla: {tempo_tupla:.4f}s")
print(f"Lista: {tempo_lista:.4f}s")
print(f"Tupla é {tempo_lista/tempo_tupla:.1f}x mais rápida")
```

Quando Usar Cada Uma?

Use Tuplas Quando:

- Dados não devem ser alterados
- Performance é crítica
- Precisa usar como chave
- Retornar múltiplos valores
- Configurações fixas

Use Listas Quando:

- Dados podem mudar
- Precisa adicionar/remover
- Coleção dinâmica
- Múltiplas operações
- Ordem pode mudar

Exemplo Prático: Sistema de Coordenadas

```
# Tupla: Coordenadas fixas de um ponto
ponto_a = (10, 20)
ponto_b = (30, 40)
```



```
# Lista: Pontos que podem ser adicionados
caminho = [ponto_a, ponto_b]
caminho.append((50, 60)) # Adicionando novo ponto
caminho.remove(ponto_a)  # Removendo ponto

# Tupla: Configuração fixa
config = ("localhost", 8080, True)

# Lista: Logs que crescem
logs = ["Erro 1", "Erro 2"]
logs.append("Erro 3") # Adicionando novo log
```

💡 Dica de Ouro

Regra prática: Se você não precisa modificar os dados após a criação, use tupla. Se precisa de flexibilidade para modificar, use lista.

